

SOLID Принципи, Builder и Strategy Design Patterns

Семинар 16
Серхан

SOLID - набор от пет принципа за обектно-ориентиран дизайн. Цел: код, лесен за разбиране, разширяване и поддръжка.

1. Single Responsibility Principle (SRP) - клас има само една причина за промяна. Клас отговаря за конкретна задача.
2. Open/Close Principle (OCP) - клас е отворен за разширяване, затворен за промяна. Нова функционалност чрез наследяване.
3. Liskov Substitution Principle (LSP) - наследник може да замени базовия клас, без да гутни логиката.
4. Interface Segregation Principle (ISP) - много малки интерфейси вместо 1 голям.
5. Dependency Inversion Principle (DIP) - класовете зависят от абстракция.

Builder Design Pattern - конструирането на сложен обект се разделя от неговото представяне.задължителните параметри са в конструктора на Builder-а, незаадължителните - чрез именовани методи. Накрая build() създава финалния обект.

Strategy Design Pattern - група алгоритми се ^{инкапсули}рат в отделни класове с общ интерфейс. Алгоритъмът може да се сменя по runtime без промяна на класа, който го използва.

SOLID - S - един клас, една отговорност
O - отворен за разширяване, затворен за промяна
L - наследникът замества базовия клас без изненади
I - много малки интерфейси, вместо един голям
D - зависи от абстракции, не определени типове

Принципите си взаимодействат:

- SRP → класовете са фокусирани
- OCP → използва полиморфизъм (наследяване + virtual)
- LSP → гарантира, че полиморфизмът работи правилно
- ISP → интерфейсите са малки и конкретни
- DIP → класовете зависят от интерфейсите, не от конкретни ^{ни} типове

Builder Design Pattern

Пример: При клас с 10 параметри - 2 задължителни, 8 опционални.

Проблем: Не знаем кой параметър какво означава

Бъркаме параметрите от един тип

Нов параметър → нов конструктор

Решение: Builder - задължителните параметри в Builder конструктора, незадължителните чрез именовани методи.

Клас/Struct Builder - вградена в класа, който я използва

↓
има същите параметри:

1 конструктор за задължителните и сетъри за незадължителните
↓
вързват Builder &

Strategy Design Pattern

Пример: Онлайн магазин с различни начини за изчисляване на отстъп-
ка

Без Strategy: при нова отстъпка → редактиране Order и за нов тип
добавяме логика

В strategy: нов алгоритъм → нов клас

IStrategy (интерфейс)
└ execute()

↑
Strategy1 Strategy2 Strategy3

Context

└ unique_ptr<IStrategy> strategy
└ setStrategy(unique_ptr<IStrategy>)
└ doWork() → strategy → execute()